

Bài Tập Cuối Chương

Tối Ưu Hóa Ứng Dụng Danh Sách Sản Phẩm

Mục tiêu:

- Áp dụng `React.lazy`, `Suspense`, `React.memo`, `useCallback`, `useMemo` vào ứng dụng duyệt sản phẩm 2 trang.
- Sử dụng React DevTools Profiler để đo lường hiệu quả tối ưu hóa trên trang danh sách sản phẩm.
- Tập trung vào các kỹ thuật tối ưu cốt lõi trong một kịch bản đơn giản hơn.

Yêu cầu:

1. **Tạo dự án:** Sử dụng Vite:

```
npm create vite@latest my-complete-optimized-products --template react
cd my-complete-optimized-products
npm install
npm install react-router-dom
```

2. **Tạo cấu trúc thư mục và file:** Tạo chính xác các thư mục và file như bên dưới.

```
src/
├── components/
│   ├── NavBar/
│   │   ├── NavBar.jsx
│   │   └── NavBar.css # File CSS
│   ├── ProductItem/
│   │   ├── ProductItem.jsx
│   │   └── ProductItem.css # File CSS
│   ├── ProductList/
│   │   ├── ProductList.jsx
│   │   └── ProductList.css # File CSS
│   └── CategoryFilter/
│       ├── CategoryFilter.jsx
│       └── CategoryFilter.css # File CSS
├── screens/
│   ├── LandingPage/
│   │   ├── LandingPage.jsx
│   │   └── LandingPage.css # File CSS
│   └── ProductsPage/
│       ├── ProductsPage.jsx
│       └── ProductsPage.css # File CSS
├── data/
│   └── products.js
├── App.jsx
├── main.jsx
└── index.css # File CSS global
```

3. **Cài đặt và sử dụng React DevTools extension.**

Mã Nguồn Ban Đầu:

- `src/index.css` (CSS Global):

```
/* src/index.css */
body {
  font-family: sans-serif;
  margin: 0;
```

```
padding: 0;
box-sizing: border-box;
background-color: #f9f9f9;
color: #333;
}

*, *:before, *:after {
  box-sizing: inherit;
}

button {
  cursor: pointer;
  padding: 8px 12px;
  border: 1px solid #ccc;
  border-radius: 4px;
  background-color: #eee;
  transition: background-color 0.2s ease;
}

button:hover:not(:disabled) {
  background-color: #ddd;
}

button:disabled {
  cursor: not-allowed;
  opacity: 0.6;
}

select {
  padding: 8px;
  border-radius: 4px;
  border: 1px solid #ccc;
}

input[type="text"] {
  padding: 8px;
  border-radius: 4px;
  border: 1px solid #ccc;
  margin-right: 5px;
}

ul {
  list-style: none;
  padding: 0;
}

a {
  text-decoration: none;
  color: #007bff;
}

a:hover {
  text-decoration: underline;
}

h1, h2, h3, h4 {
  margin-top: 0;
}

hr {
```

```
border: none;
border-top: 1px solid #eee;
margin: 20px 0;
}
```

- `src/data/products.js`

```
// src/data/products.js
let productIdCounter = 1;
export const getProducts = () => {
  console.log("--- Generating Products ---"); // Giả lập việc tạo/fetch dữ liệu
  return [
    { id: productIdCounter++, name: 'Laptop Pro 14"', category: 'Electronics', price: 1499 },
    { id: productIdCounter++, name: 'Wireless Mouse', category: 'Electronics', price: 49 },
    { id: productIdCounter++, name: 'Organic Cotton T-Shirt', category: 'Apparel', price: 29 },
    { id: productIdCounter++, name: 'Ceramic Coffee Mug', category: 'Home Goods', price: 15 },
    { id: productIdCounter++, name: 'Running Shoes X', category: 'Apparel', price: 120 },
    { id: productIdCounter++, name: 'Smart Speaker Mini', category: 'Electronics', price: 59 },
    { id: productIdCounter++, name: 'Classic Blue Jeans', category: 'Apparel', price: 60 },
    { id: productIdCounter++, name: 'LED Desk Lamp', category: 'Home Goods', price: 35 },
    { id: productIdCounter++, name: 'Backlit Keyboard', category: 'Electronics', price: 75 },
    { id: productIdCounter++, name: 'Wool Blend Hoodie', category: 'Apparel', price: 85 },
    { id: productIdCounter++, name: 'Insulated Water Bottle', category: 'Home Goods', price: 25 },
    { id: productIdCounter++, name: '4K Monitor 27"', category: 'Electronics', price: 350 },
    { id: productIdCounter++, name: 'Graphic Tablet', category: 'Electronics', price: 99 },
    { id: productIdCounter++, name: 'Summer Dress', category: 'Apparel', price: 70 },
    { id: productIdCounter++, name: 'Scented Candle Set', category: 'Home Goods', price: 30 },
  ];
};

export const getCategories = (products) => {
  return ['All Categories', ...new Set(products.map(p => p.category))];
};
```

- `src/main.jsx`

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App.jsx';
import './index.css'; // Import CSS global

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
);
```

- `src/components/NavBar/NavBar.jsx`

```
import React from 'react';
import { Link } from 'react-router-dom';
import './NavBar.css'; // Import CSS

function NavBar() {
  console.log("Rendering NavBar");
  return (
    <nav className="NavBar">
      <Link to="/" className="NavBar-link">Home</Link>
      <Link to="/products" className="NavBar-link">Products</Link>
    </nav>
  );
}
```

```
    </nav>
  );
}

export default NavBar; // Tạm thời chưa memo
```

- src/components/NavBar/NavBar.css

```
/* src/components/NavBar/NavBar.css */
.NavBar {
  background-color: #f0f0f0;
  padding: 15px 20px;
  margin-bottom: 20px;
  border-bottom: 1px solid #ddd;
}

.NavBar-link {
  margin-right: 15px;
  text-decoration: none;
  color: #333;
  font-weight: bold;
  transition: color 0.2s ease;
}

.NavBar-link:hover {
  color: #007bff;
}
```

- src/components/ProductItem/ProductItem.jsx

```
import React from 'react';
import './ProductItem.css'; // Import CSS

// Ban đầu chưa dùng React.memo
function ProductItem({ product, onAddToCart }) {
  // console.log(`Rendering ProductItem: ${product.id} - ${product.name}`); // Sẽ thêm ở bước tối ưu
  return (
    <li className="ProductItem">
      <div className="ProductItem-details">
        <h4 className="ProductItem-name">{product.name}</h4>
        <p className="ProductItem-category">Category: {product.category}</p>
        <p className="ProductItem-price">Price: ${product.price}</p>
      </div>
      <button className="ProductItem-button" onClick={() => onAddToCart(product.id)}>
        Add to Cart
      </button>
    </li>
  );
}

export default ProductItem;
```

- src/components/ProductItem/ProductItem.css

```
/* src/components/ProductItem/ProductItem.css */
.ProductItem {
  border: 1px solid #ddd;
  border-radius: 4px;
  margin-bottom: 15px; /* Thay margin thành margin-bottom */
}
```

```
padding: 15px;
list-style: none;
display: flex;
justify-content: space-between;
align-items: center;
background-color: #fff;
}

.ProductItem-details {
  flex-grow: 1;
  margin-right: 15px;
}

.ProductItem-name {
  margin: 0 0 5px 0;
  color: #333;
}

.ProductItem-category {
  margin: 0 0 5px 0;
  font-size: 0.9em;
  color: #555;
}

.ProductItem-price {
  margin: 0;
  font-size: 1.1em;
  font-weight: bold;
  color: #28a745; /* Màu giá tiền */
}

.ProductItem-button {
  padding: 8px 15px;
  flex-shrink: 0; /* Ngăn nút bị co lại */
  background-color: #007bff;
  color: white;
  border: none;
}

.ProductItem-button:hover {
  background-color: #0056b3;
}
```

- `src/components/ProductList/ProductList.jsx`

```
import React from 'react';
import ProductItem from '../ProductItem/ProductItem';
import './ProductList.css'; // Import CSS

function ProductList({ products, onAddToCart }) {
  console.log("Rendering ProductList");
  if (!products || products.length === 0) {
    return <p className="ProductList-empty">No products found for this category.</p>;
  }
  return (
    <ul className="ProductList">
      {products.map(product => (
        <ProductItem key={product.id} product={product} onAddToCart={onAddToCart} />
      ))}
    </ul>
  );
}
```

```
    </ul>
  );
}

export default ProductList;
```

- `src/components/ProductList/ProductList.css`

```
/* src/components/ProductList/ProductList.css */
.ProductList {
  list-style: none;
  padding: 0;
}

.ProductList-empty {
  text-align: center;
  color: #777;
  margin-top: 30px;
  font-style: italic;
}
```

- `src/components/CategoryFilter/CategoryFilter.jsx`

```
import React from 'react';
import './CategoryFilter.css'; // Import CSS

// Ban đầu chưa dùng React.memo
function CategoryFilter({ categories, currentCategory, onCategoryChange }) {
  console.log("Rendering CategoryFilter");
  return (
    <div className="CategoryFilter">
      <label htmlFor="category-select" className="CategoryFilter-label">Filter by Category:</label>
      <select
        id="category-select"
        className="CategoryFilter-select"
        value={currentCategory}
        onChange={(e) => onCategoryChange(e.target.value)}
      >
        {categories.map(category => (
          <option key={category} value={category}>{category}</option>
        ))}
      </select>
    </div>
  );
}

export default CategoryFilter;
```

- `src/components/CategoryFilter/CategoryFilter.css`

```
/* src/components/CategoryFilter/CategoryFilter.css */
.CategoryFilter {
  margin-bottom: 25px;
  display: flex;
  align-items: center;
}

.CategoryFilter-label {
  margin-right: 10px;
}
```



```
font-weight: bold;
}

.CategoryFilter-select {
padding: 10px;
border-radius: 4px;
border: 1px solid #ccc;
min-width: 200px; /* Đảm bảo độ rộng tối thiểu */
}
```

- `src/screens/LandingPage/LandingPage.jsx`

```
import React from 'react';
import { Link } from 'react-router-dom';
import './LandingPage.css'; // Import CSS

function LandingPage() {
  console.log("Rendering LandingPage");
  return (
    <div className="LandingPage">
      <h1>Welcome to Our Product Catalog!</h1>
      <p>Explore our amazing products. This app is designed for practicing React optimization techniques.</p>
      <Link to="/products">
        <button className="LandingPage-button">
          View Products
        </button>
      </Link>
    </div>
  );
}

export default LandingPage;
```

- `src/screens/LandingPage/LandingPage.css`

```
/* src/screens/LandingPage/LandingPage.css */
.LandingPage {
padding: 40px 20px;
text-align: center;
max-width: 600px;
margin: 40px auto;
background-color: #fff;
border-radius: 8px;
box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
}

.LandingPage h1 {
color: #333;
margin-bottom: 15px;
}

.LandingPage p {
color: #555;
line-height: 1.6;
margin-bottom: 30px;
}

.LandingPage-button {
```

```
padding: 12px 25px;
font-size: 1.1em;
cursor: pointer;
background-color: #28a745;
color: white;
border: none;
border-radius: 5px;
transition: background-color 0.2s ease;
}

.LandingPage-button:hover {
  background-color: #218838;
}
```

- `src/screens/ProductsPage/ProductsPage.jsx`

```
import React, { useState, useMemo } from 'react';
import ProductList from '../components/ProductList/ProductList';
import CategoryFilter from '../components/CategoryFilter/CategoryFilter';
import { getProducts, getCategories } from '../data/products';
import './ProductsPage.css'; // Import CSS

function ProductsPage() {
  console.log("Rendering ProductsPage");
  const [allProducts, setAllProducts] = useState(() => getProducts());
  const [selectedCategory, setSelectedCategory] = useState('All Categories');
  const [cartItems, setCartItems] = useState([]);
  const [counter, setCounter] = useState(0); // State giả để test re-render

  const categories = useMemo(() => getCategories(allProducts), [allProducts]);

  // Logic lọc (chưa tối ưu bằng useMemo)
  const filteredProducts = allProducts.filter(product => {
    console.log("Filtering logic running...");
    return selectedCategory === 'All Categories' || product.category === selectedCategory;
  });

  // Hàm xử lý (chưa tối ưu bằng useCallback)
  const handleCategoryChange = (newCategory) => {
    setSelectedCategory(newCategory);
  };

  const handleAddToCart = (productId) => {
    console.log(`Adding product ${productId} to cart`);
    if (!cartItems.includes(productId)) {
      setCartItems([...cartItems, productId]);
    } else {
      console.log(`Product ${productId} already in cart.`);
    }
  };

  const handleRefreshProducts = () => {
    console.log("Refreshing product list...");
    setAllProducts(getProducts().map(p => ({ ...p, refreshId: Date.now() })));
  };

  return (
    <div className="ProductsPage">
      <h1>Product Catalog</h1>
```



```

    <div className="ProductsPage-controls">
      <button onClick={handleRefreshProducts}>Refresh Product List</button>
      <p className="ProductsPage-cartInfo">Items in Cart: {cartItems.length}</p>
      <button onClick={() => setCounter(c => c + 1)}>
        Force Re-render (Counter: {counter})
      </button>
    </div>
    <CategoryFilter
      categories={categories}
      currentCategory={selectedCategory}
      onCategoryChange={handleCategoryChange}
    />
    <ProductList products={filteredProducts} onAddToCart={handleAddToCart} />
  </div>
);
}

export default ProductsPage;

```

- `src/screens/ProductsPage/ProductsPage.css`

```

/* src/screens/ProductsPage/ProductsPage.css */
.ProductsPage {
  padding: 20px;
  max-width: 900px; /* Giới hạn chiều rộng */
  margin: 0 auto; /* Căn giữa */
}

.ProductsPage h1 {
  text-align: center;
  margin-bottom: 25px;
  color: #444;
}

.ProductsPage-controls {
  margin-bottom: 25px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #f8f9fa;
  padding: 15px;
  border-radius: 5px;
  border: 1px solid #eee;
}

.ProductsPage-controls button {
  padding: 10px 15px;
}

.ProductsPage-cartInfo {
  margin: 0;
  font-weight: bold;
}

```

- `src/App.jsx`

```

import React from 'react'; // Thêm { Suspense } ở Bước 1
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import NavBar from './components/NavBar/NavBar';

```

```
// Import trực tiếp ban đầu - Sẽ đổi thành React.lazy ở Bước 1
import LandingPage from './screens/LandingPage/LandingPage';
import ProductsPage from './screens/ProductsPage/ProductsPage';

function App() {
  return (
    <BrowserRouter>
      <NavBar />
      { /* <React.Suspense fallback={<div style={{padding: '40px', textAlign: 'center', fontSize: '1.2em'}}>Loading page content...</div>}> */ }
      <Routes>
        <Route path="/" element={<LandingPage />} />
        <Route path="/products" element={<ProductsPage />} />
        { /* Route 404 đơn giản */ }
        <Route path="*" element={<div style={{padding: '40px', textAlign: 'center'}}><h2>404 - Page Not Found</h2></div>} />
      </Routes>
    { /* </React.Suspense> */ }
  </BrowserRouter>
);
}

export default App;
```

Các Bước Thực Hiện Tối Ưu

Bước 0: Chuẩn bị và Đo lường Baseline

- Chạy Ứng Dụng:** Khởi động dự án bằng `npm run dev` và mở trong trình duyệt.
- Mở Công Cụ:** Mở Developer Tools và React DevTools (tab Components & Profiler).
- Bắt Đầu Ghi (Profiler):** Nhấn nút "Record".
- Thực Hiện Các Hành Động:** Tương tác với ứng dụng theo chuỗi:
 - Truy cập trang Landing (/).
 - Điều hướng đến trang Products (/products).
 - Thay đổi bộ lọc Category nhiều lần.
 - Thêm vài sản phẩm vào giỏ hàng.
 - Nhấn nút "Refresh Product List".
 - Nhấn nút "Force Re-render" vài lần.
- Dừng Ghi (Profiler):** Nhấn nút "Stop".
- Phân Tích Baseline:**
 - Profiler:** Xem lại bản ghi, chú ý số lần render và thời gian render của các component chính (ProductsPage , ProductList , ProductItem , CategoryFilter) trong các tình huống tương tác khác nhau (đặc biệt khi lọc, thêm vào giỏ, và force re-render).
 - Console:** Xem lại logs, đặc biệt log "Rendering..." từ các component và log "Filtering logic running...". Lưu ý khi nào chúng xuất hiện.
 - Ghi Chú:** Ghi lại các quan sát về hiệu năng và số lần render không cần thiết để làm cơ sở so sánh.

Bước 1: Tối ưu Tải Trang với React.lazy và Suspense

- Mở File:** Chỉnh sửa file `src/App.jsx` .
- Import Công Cụ:** Đảm bảo `Suspense` được import từ `react` .
- Chuyển đổi Imports:** Thay đổi cách import các component màn hình (LandingPage , ProductsPage) từ dạng import tĩnh sang dynamic import sử dụng `React.lazy` .

4. **Áp Dụng `Suspense`** : Bọc component `<Routes>` bằng component `<Suspense>` . Cung cấp một giao diện JSX hợp lệ cho prop `fallback` của `<Suspense>` (ví dụ: một thẻ `div` với text "Loading...").
5. **Kiểm Tra:**
- Lưu file, đảm bảo ứng dụng chạy ổn.
 - Mở tab "Network" trong DevTools, disable cache.
 - Tải lại trang (`/`). Quan sát các file JS ban đầu.
 - Điều hướng đến `/products` . Quan sát xem có một file chunk JS mới được tải không.

Bước 2: Ngăn Chặn Re-render Không Cần Thiết với `React.memo`

1. **Tối ưu `ProductItem`** :
- Mở file `src/components/ProductItem/ProductItem.jsx` .
 - Thêm một lệnh `console.log` vào đầu function để theo dõi việc render của nó (bao gồm cả `product.id` hoặc `product.name` trong log).
 - Import `React` .
 - Sử dụng Higher-Order Component `React.memo` để bọc định nghĩa của function component `ProductItem` .
2. **Tối ưu `CategoryFilter`** :
- Mở file `src/components/CategoryFilter/CategoryFilter.jsx` .
 - Import `React` .
 - Sử dụng `React.memo` để bọc định nghĩa của function component `CategoryFilter` .
3. **(Tùy chọn) Tối ưu `NavBar`** : Thực hiện tương tự như trên cho component `NavBar` .
4. **Quan Sát và Đo Lường:**
- Lưu các thay đổi, quay lại ứng dụng và mở Console.
 - Trên trang `/products` :
 - Thực hiện các hành động: Thay đổi Category, "Add to Cart", "Refresh Product List", "Force Re-render".
 - Quan sát Console: Log từ `ProductItem` và `CategoryFilter` có xuất hiện trong các trường hợp mà bạn kỳ vọng chúng *không* nên re-render không (ví dụ: khi thêm item khác vào giỏ, khi force re-render cha)?
 - **Sử dụng Profiler:** Ghi lại các hành động này và so sánh số lần render của `ProductItem` và `CategoryFilter` với baseline (Bước 0).

Bước 3: Tối Ưu Hóa Logic Lọc Sản Phẩm với `useMemo`

1. **Mở File:** Chỉnh sửa file `src/screens/ProductsPage/ProductsPage.jsx` .
2. **Import Hook:** Đảm bảo `useMemo` được import từ `react` .
3. **Xác định Tính Toán:** Tìm đoạn code tính toán biến `filteredProducts` .
4. **Áp dụng `useMemo`** : Sử dụng hook `useMemo` để bọc hàm thực hiện logic lọc sản phẩm. Cung cấp chính xác mảng dependencies cho `useMemo` (những giá trị mà khi thay đổi sẽ yêu cầu tính toán lại bộ lọc). Thêm một `console.log` *bên trong* hàm tạo của `useMemo` để biết khi nào nó thực sự chạy lại.
5. **Quan Sát:**
- Lưu file, quay lại ứng dụng và mở Console.
 - Thực hiện các hành động: Thay đổi Category, "Add to Cart", "Refresh Product List", "Force Re-render".
 - Quan sát log mới từ bên trong `useMemo` : Nó chỉ nên xuất hiện khi dependencies (ví dụ: danh sách sản phẩm gốc hoặc category được chọn) thay đổi. Nó *không* nên xuất hiện khi chỉ thêm vào giỏ hàng hoặc khi nhấn nút force re-render.

Bước 4: Ổn Định Callbacks với `useCallback`

1. **Mở File:** Vẫn trong `src/screens/ProductsPage/ProductsPage.jsx` .
2. **Import Hook:** Đảm bảo `useCallback` được import từ `react` .
3. **Xác định Hàm:** Tìm các định nghĩa hàm `handleCategoryChange` và `handleAddToCart` .

4. **Áp dụng `useCallback`** : Sử dụng hook `useCallback` để bọc định nghĩa của từng hàm này. Cung cấp chính xác mảng dependencies cho mỗi `useCallback` (những giá trị từ scope bên ngoài mà hàm đó sử dụng - lưu ý rằng các hàm setter từ `useState` có tham chiếu ổn định). Cân nhắc sử dụng functional update bên trong callback nếu nó cập nhật state dựa trên state trước đó.

5. Quan Sát và Đo Lường:

- Lưu file và quay lại ứng dụng.
- **Sử dụng Profiler:** Ghi lại các hành động, đặc biệt là nhấn nút "Force Re-render" nhiều lần trên trang `/products`.
- **Phân Tích Profiler:** So sánh với kết quả Profiler ở Bước 2. Sau khi áp dụng `useCallback` cho các hàm truyền xuống props, các component con đã memo (`ProductItem` , `CategoryFilter`) có còn bị re-render khi nhấn nút "Force Re-render" không? (Lý tưởng là không).

Bước 5: Đánh giá Tổng thể

1. **Đo Lường Lại:** Sử dụng **Profiler** một lần nữa, thực hiện lại *toàn bộ* chuỗi hành động như đã mô tả ở Bước 0.

2. So Sánh Cuối Cùng:

- Đối chiếu bản ghi Profiler cuối cùng này với bản ghi baseline ban đầu (Bước 0).
- Đánh giá xem số lần render không cần thiết của các component đã giảm chưa.
- Kiểm tra xem logic tính toán (lọc) có chạy ít lần hơn không.
- Xem xét liệu có sự cải thiện nào về hiệu năng cảm nhận được không.